Parte Teórica — Trabalho Web Development Front-End

Disciplina: Web Programming Front-End

Tema: Criação de uma aplicação front-end para uma API de desenhos de sucesso dos anos 90 e 2000

API utilizada: Rick and Morty API (https://rickandmortyapi.com/)

Limite: Até 3 páginas

---

1. O que é uma API e como funciona o consumo de dados em tempo real

Uma **API** (Application Programming Interface, ou Interface de Programação de Aplicações) é um conjunto de regras e protocolos que permitem que dois sistemas de software se comuniquem entre si. No contexto do desenvolvimento web, uma API funciona como um "garçom": ela recebe a requisição do cliente (navegador), leva ao servidor, e traz de volta a resposta com os dados solicitados.

O consumo de dados em **tempo real** ocorre quando a aplicação front-end faz requisições assíncronas a essa API, geralmente através do JavaScript, e atualiza a interface da página sem precisar recarregar todo o site. No nosso projeto, utilizamos o método `fetch` para enviar uma requisição GET à Rick and Morty API, que responde com um objeto JSON contendo a lista de personagens. Esse processo é assíncrono: o navegador continua executando outras tarefas enquanto aguarda a resposta do servidor, e quando os dados chegam, a página é atualizada dinamicamente.

---

2. Conceito de DOM Manipulation e criação de elementos dinâmicos via JavaScript

O **DOM** (Document Object Model) é uma representação em árvore de todos os elementos HTML de uma página. O navegador cria esse modelo ao carregar o HTML, permitindo que o JavaScript acesse, modifique, adicione ou remova elementos da página.

DOM Manipulation é a técnica de alterar a estrutura, o estilo ou o conteúdo de uma página web dinamicamente usando JavaScript. Em vez de escrever todos os cards de personagens diretamente no HTML (o que seria estático e impraticável), criamos os elementos através de JavaScript com métodos como:

- `document.createElement('tag')` — cria um novo elemento HTML

- `element.appendChild(filho)` — insere um elemento dentro de outro

- `element.textContent` — define o texto de um elemento

- `element.setAttribute('atributo', 'valor')` — define atributos como `src` de imagem

No nosso projeto, ao receber os dados da API, percorremos a lista de personagens com `forEach` e, para cada um, criamos um `<article>` (card), uma imagem, um `<h3>` (nome), um `<span>` (status) e um `<p>` (espécie), inserindo tudo no container da página. Isso garante que os cards sejam gerados dinamicamente com base nos dados reais da API.

---

3. Explicação das funções básicas utilizadas para consumo da API

As principais funções e métodos utilizados no projeto são:

3.1 `fetch()`

É uma função nativa do JavaScript que faz requisições HTTP de forma assíncrona. Retorna uma `Promise` que resolve para um objeto `Response`. No projeto, usamos:

```
const response = await fetch(url, {
   method: 'GET',
   mode: 'cors',
   headers: { 'Content-Type': 'application/json' }
});
```

3.2 `async / await`

Permite escrever código assíncrono de forma síncrona e legível. `await` pausa a execução até que a Promise seja resolvida, facilitando o tratamento da resposta da API.

3.3 `.json()`

Método do objeto `Response` que converte o corpo da resposta (geralmente em formato JSON) em um objeto JavaScript manipulável.

3.4 `createElement()`

Cria um novo elemento HTML no DOM. Exemplo:

```
const card = document.createElement('article');
```

3.5 `appendChild()`

Insere um elemento filho dentro de um elemento pai no DOM. Exemplo:

```
containerCards.appendChild(card);
```

3.6 `addEventListener()`

Associa um evento (como clique ou digitação) a um elemento, permitindo interatividade com o usuário.

---

4. Motivo da escolha da API

Foi escolhida a **Rick and Morty API** pelos seguintes motivos:

- **Gratuita e pública:** Não requer cadastro, chave de API (token) nem pagamento para acesso.
- **Tema alinhado:** Rick and Morty é uma série de animação de sucesso dos anos 2000, encaixando perfeitamente no tema do trabalho ("desenhos de sucesso dos anos 90 e 2000").
- **Dados completos:** Fornece nome, status (vivo/morto/desconhecido), espécie, gênero, imagem e localização de cada personagem.
- **Estável e bem documentada:** Possui documentação clara e endpoint simples para listagem e paginação de personagens.
- **Suporte a CORS:** Permite requisições diretamente do navegador sem problemas de política de mesma origem.

---

5. Regras de acesso e documentação da API utilizada

5.1 Endpoint principal

```
https://rickandmortyapi.com/api/character
```

5.2 Regras de acesso

- **Não requer autenticação:** Não é necessário token, cadastro ou chave de API.
- **CORS habilitado:** Pode ser consumida diretamente pelo navegador via JavaScript.
- **Método permitido:** GET para listagem de personagens.
- **Paginação:** A API retorna 20 resultados por página. Os parâmetros `?page=N` e `?name=busca` permitem navegar e filtrar.
- **Limite de requisições:** A API não divulga um limite rígido público, mas recomenda-se uso razoável para evitar bloqueios temporários.

5.3 Estrutura da resposta (JSON)

A resposta da API segue este formato simplificado:

```
{
 "info": {
  "count": 826,
  "pages": 42,
  "next": "https://rickandmortyapi.com/api/character/?page=2",
  "prev": null
 },
 "results": [
  {
   "id": 1,
   "name": "Rick Sanchez",
   "status": "Alive",
   "species": "Human",
   "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
   "gender": "Male",
   "origin": { "name": "Earth (C-137)" },
   "location": { "name": "Earth (C-137)" }
  }
 ]
}
```


5.4 Documentação oficial

A documentação completa está disponível em: https://rickandmortyapi.com/documentation

---

Referências

- MDN Web Docs — Using Fetch API: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API/Using_Fetch

- MDN Web Docs — Document.createElement: https://developer.mozilla.org/pt-BR/docs/Web/API/Document/createElement

- MDN Web Docs — Node.appendChild: https://developer.mozilla.org/pt-BR/docs/Web/API/Node/appendChild

- Rick and Morty API: https://rickandmortyapi.com/